One Operator, Larger Graphs: Learning to Compare Symbolic Expressions

Sahil Singh*
Fulton Science Academy Private School
Georgia Institute of Technology

Quang Bui
American International School Vienna

Muhammad Rayyan Chand
Yasmina British Academy

Sibi Gokul
National Institute of Technology Warangal

Daksh Jain
Broward College

Charvik Morampudi
Georgia Institute of Technology

Abstract

Can rewriting mathematics with a single operation make it easier for a neural model to compare expressions? We separate three questions: whether the expressions become smaller, whether a model learns its training examples, and whether it succeeds on new ones. We compare source expression graphs with two versions compiled into $E(a, b) = \exp(a) - \log b$ (EML). Recorded summaries for one fixed compiler show a median 41-fold expansion in expression-tree size. Within one graph-model family, all three representations can fit the training examples by the final update yet remain near chance on variable swaps and unfamiliar expression patterns. The tests use different examples, so the drop alone does not identify its cause. To probe the role of training data, we change the examples used to train and choose models. Source-graph accuracy on fresh swaps of familiar patterns rises from about 50% to 92%; unfamiliar patterns remain near chance. Both EML versions still fail to fit the changed training examples within the fixed budget, preventing a clean test of their generalization. Removing the bounded-EML cap on node-distance features also leaves its original swap test near chance. A separate compiler for rational polynomials on bounded variable ranges keeps calculations in their real domains in exact arithmetic; numerical checks support, but do not formally verify, its implementation. Neural equivalence scores remain suggestions for an independent checker, not proofs.

1 Introduction

The expressions $x + y$ and $y + x$ look different but compute the same function. A learned equivalence model tries to recognize such relationships from examples [1]. Equivalent pairs are positives; non-equivalent pairs are negatives. A variable swap exchanges two variable names throughout an expression: $2x + y$ becomes $2y + x$, with the same shape and coefficients but different values when $x \neq y$. Distinguishing them requires variable binding: tracking which variable receives each coefficient.

Fitting means succeeding on the examples used to train the model; generalization means succeeding on new examples. Both can depend on representation: multiplication might occupy one node in an expression graph or a long sequence of more basic operations.

*Corresponding author: gensahilsingh@gatech.edu

NeurIPS 2026 Workshop on AI for Verifiable Coding.

Odrzywołek [2] shows that a single operation,

$$E(a, b) = \exp(a) - \log b, \tag{1}$$

together with the constant one, can express a scientific-calculator basis. We call this representation EML. This does not establish compiled size, whether intermediate calculations remain in the intended domain, or how easily a model can learn from the representation.

These questions matter for verifiable coding: equality saturation stores alternative rewrites in a shared structure [3, 4], while learned models can guide proof search [5, 6]. A small primitive vocabulary can move useful information into long graph paths. High generated-data accuracy can conceal failures under new compositions [7] or benchmark shortcuts [8]. We study a representation choice for such systems, without evaluating a proof assistant or code generator. A sound pipeline still requires an independent symbolic checker to accept each proposed step.

We measure one EML compiler's structural costs using recorded summaries for 250,000 generated expressions and recounting graphs for a separate 2,700-expression learning population. Changing binding-sensitive examples raises source-graph swap accuracy on familiar expression patterns from 50% to 92%, while EML models underfit; removing a cap on bounded-EML node distances leaves the original swap test near chance. A separate real-valued construction covers rational-coefficient polynomials on bounded variable ranges.

Neural symbolic mathematics includes expression transformation [9], generated reasoning tasks [10], and arithmetic extrapolation [11]. Symbolic regression instead recovers formulas from observations [12, 13]. Our contribution is EML-specific cost and learning evidence, not a new general theory of learned equivalence.

2 Representations and study design

An expression tree has one node for each operation, variable, or constant occurrence. A shared expression graph stores repeated subexpressions once, using established term-graph and hash-consing ideas [14, 15]. Such a graph is a directed acyclic graph (DAG): following its dependencies never returns to the starting node. We compare three inputs:

- **Source expression graph (AST).** Derived from an abstract syntax tree, it keeps familiar operations such as addition and multiplication explicit and shares repeated subexpressions.
- **Historical EML.** Our implementation of the reference v4 formulas [16], used for the 250,000-expression corpus, rewrites these operations into E and shares repeated terms.
- **Bounded EML.** A separate compiler handles exact rational polynomials over declared rational variable bounds. Its real-valued construction is described below. It changes graph structure as well as domain behavior.

EML stores coefficient magnitudes in primitive graph structure; it does not add the source coefficients as extra scalar features.

Each generator template is an expression pattern, such as commutativity. A context fixes the template, coefficients, and variable names; literals are numerical constants written in an expression.

Position features give each node two distances: the shortest path from the output and the longest path from an input leaf. Clipping replaces distances above a cap with that cap. A saved model state stores the learned weights. The selected state has the lowest prediction loss on separate validation examples; the final state is saved at the last training update.

Exact polynomial calculation supplies the labels: positive pairs have identical rational coefficients, while every negative has an assignment where its values differ. This verifies source labels; numerical agreement of compiled graphs at tested assignments does not prove agreement everywhere.

A real-valued construction on a restricted domain. The bounded compiler accepts exact integer and rational constants, variables, addition, and multiplication. Fixed nonnegative integer powers are written as repeated multiplication. The experiments use six variables, each between $1/4$ and 4; the construction extends to other declared rational bounds. It does not cover transcendental source expressions.

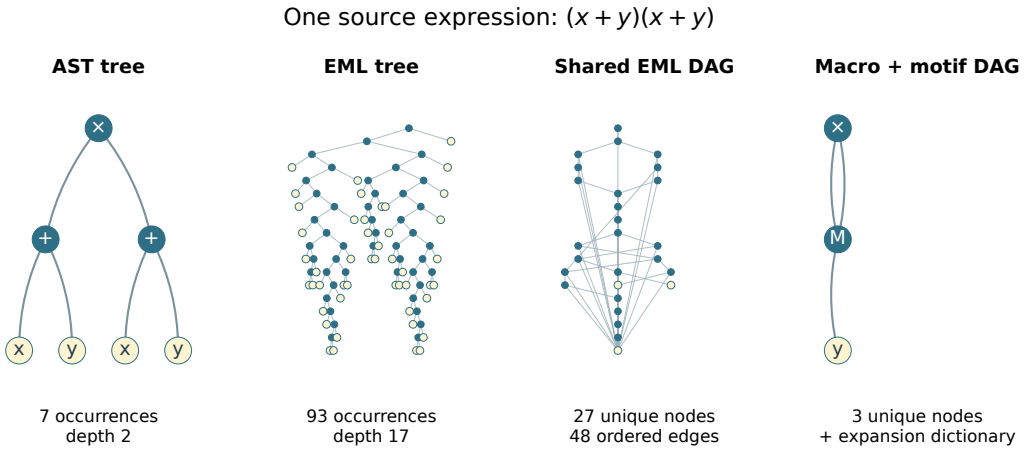


Figure 1: A small expression can become a much larger compiled graph. For $(x + y)(x + y)$, the source tree has seven occurrences and its shared graph has four nodes. Historical EML has 93 tree occurrences and 27 shared nodes. A three-node motif view hides repeated patterns behind a dictionary, whose definitions must also be counted. This example uses the historical compiler.

Table 1: Which experiment supports which claim. The large structural population and the smaller learning populations are separate.

Population	Inputs and model	Model choice / semantics	Supported claim
250,000 expressions	Source and historical EML; no learner	Fixed compiler; no semantic guarantee needed for counting	Recorded structural expansion
32-pair diagnostics	Source and historical EML; local and positional models	Training-only controls; old EML domain limits remain	Architecture-dependent fitting and measured signal size
32 training, 32 validation pairs	All three graphs; positional models	Variable-swap negatives; three seeds	Fitting on small binding-sensitive controls
480 training, 120 validation; three seeds	All three graphs; twelve positional layers	Selected models primary; final models diagnostic; bounded EML from restricted-domain compiler	Fitting, model selection, and performance on two 600-pair tests
Follow-up interventions (after original results)			
480 training, 120 validation; nine fits	All three graphs; same twelve-layer recipe	Half of train/validation negatives replaced; two fresh 600-pair panels	Response to one binding-sensitive population intervention
480 training, 120 validation; six fits	Bounded EML; position caps 63/255	Matched table size; selected primary, final secondary	Sensitivity to positional clipping under one recipe

The main idea is to keep every logarithm's argument positive. For example, $E(t, 1) = \exp(t)$, and a short composition gives $\log(t)$ when $t > 0$. Exact interval bounds [17] tell the compiler when to add a positive offset before using a subtraction identity and remove the offset afterward. Products that cross zero can similarly be rewritten using positive shifted factors. Induction over the source expression gives value preservation and valid real-valued primitive domains, using the stated identities and exact interval bounds. Appendix B gives the construction.

All 21,107 retained polynomial expressions from an earlier dataset compile, and high-precision evaluation agrees with exact source values at three assignments and two precisions: 126,642 whole-graph evaluations, each checking every visited primitive's domain. Equal-coordinate assignments cannot reveal variable-swap errors, so a uniform audit also evaluates every saved bounded graph in the six original/intervention learning populations at an unequal-coordinate counterexample. All

10,800 evaluations agree and every negative remains numerically distinct (Appendix C). These finite checks do not verify the compiler; finite values can still be very large or poorly conditioned [18].

3 How much larger are the graphs?

The large structural population. The 250,000-expression population was generated with seed 20260721 and fixed quotas spanning algebraic, rational, exponential/logarithmic, trigonometric/hyperbolic, mixed, and stress expressions. Source trees have median 29 nodes, 95th percentile 123, and range 2–213, with depths 1–17. This is a designed benchmark, not a sample of operation frequencies in practical mathematics. The supplement gives the grammar, quotas, splits, rejection counts, and generation commands. [2]

The 250,000-expression compiled-cost statistics are recorded summaries: the original per-expression ledger is unavailable. Reparsing the locally retained sources reproduces the reported source sizes; the supplement releases their per-expression measurements, rather than the full historical corpus. Table 2 and Appendix E give replayable per-expression costs for the separate 2,700 learning-study expressions. These polynomial-only inputs do not verify the larger corpus's compiled costs.

On the 250,000-expression corpus, historical tree expansion has median $40.66\times$, mean $952.14\times$, and 99th percentile $10{,}448.60\times$. The ratio of total compiled to total source tree nodes is $532.86\times$; it differs from the mean of per-expression ratios. Saved family summaries locate the largest expansions in trigonometric/hyperbolic sources (median $270.46\times$, 90th percentile $4{,}566.15\times$), compared with algebraic sources ($25.58\times$ and $33.34\times$). Sharing reduces the mean per-expression ratio to $10.47\times$, with compiled DAG nodes divided by source DAG nodes. These are fixed-compiler measurements, not lower bounds for all EML encodings.

A motif dictionary shortens the visible graph but must itself be counted [19], as Figure 1 illustrates. The learning comparison uses no dictionaries.

Table 2: Paired graph costs on the same 2,700 source expressions in the current development and two test populations. Depth is the longest dependency path. The ratio averages per-expression node ratios to the source graph.

Input	Nodes median	Nodes p95	Depth median	Depth p95	Mean ratio
Source	12	19	3	6	1.00
Historical EML	104	184	43	69	10.1
Bounded EML	249	792	103	183	28.9

The bounded compiler changes more than domain behavior: it also makes larger, deeper graphs with different sharing and position information. The comparison therefore tests two compiler configurations; it cannot isolate the effect of valid intermediate calculations.

4 What allows the models to fit?

The local graph model repeatedly combines each node's features with information from its neighbors [20]. With few rounds, information cannot travel far. Color refinement asks whether such local summaries can tell graphs apart [21, 22]: each node receives a new label summarizing its current label, its neighbors' labels (including repetitions), and edge attributes. Identical resulting graph summaries imply an obstruction for the specified model class in ideal arithmetic. Different summaries permit separation; they do not show that training will find it.

The three-layer EML model encounters this obstruction on matched examples. Twelve layers remove it without producing a successful fit. On 32 balanced development pairs, six training and graph-summary choices are applied equally to source and EML models. Five source choices pass; no EML choice does. Binary cross entropy (BCE) is a prediction loss: lower is better, and always guessing probability one-half gives approximately 0.693 on balanced data. A fitting pass requires training accuracy at least 95% and BCE at most 0.2. Tiny controls require three consecutive passing checks. Appendix A discusses possible depth-related mechanisms; neither is shown to cause the EML failure.

[2] `https://github.com/sahilsinghthefirst/GEML/releases/tag/vericodegen-camera-ready-20261014`

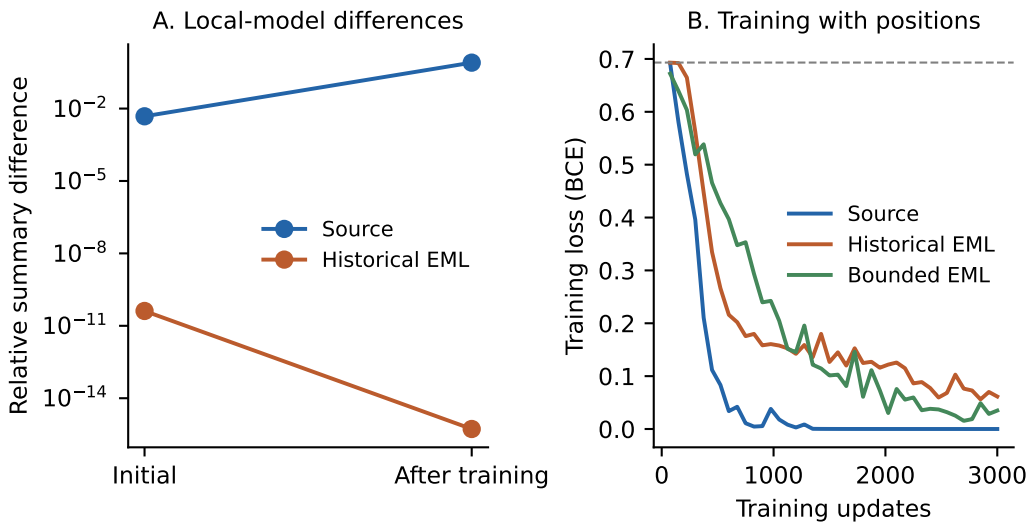


Figure 2: Two different pieces of evidence. Left: FP64 matched-summary distance divided by mean summary norm for the local three-layer model, before and after training; the trained historical-EML value is at FP64 rounding level. Right: training loss of the twelve-layer positional models on the original 480-pair population, for seed 20260921; the dashed line is constant-prediction loss. The panels use different architectures and populations. Table 4 summarizes training outcomes and fitting criteria; Appendix Table 8 lists every seed.

Small signals require further explanation. We measure how far apart the local model places matched graph summaries, relative to their overall magnitude. With a fixed 64-number summary and each model's saved weights evaluated in 64-bit arithmetic (FP64), the initial median relative difference is 4.8×10^{-3} for source graphs and 4.1×10^{-11} for historical EML. This ratio cancels common scaling. In standard 32-bit arithmetic (FP32), reversing message order changes historical-EML summaries by about as much as the FP32 matched difference; both exceed the FP64 separation by orders of magnitude. Figure 2 shows these very small distinguishing signals; they do not establish that numerical precision causes the fitting failure.

Positions make a useful difference. Adding the two whole-graph position features, capped at 63, follows the broader use of structural encodings in graph learning [23–25]. All sixteen tiny controls spanning source/EML inputs, three/twelve rounds, two seeds, and shared/separate layer weights pass. The intervention changes both information and optimization, so its success does not identify a unique mechanism.

Layerwise readouts [26] and initial residual connections [27] motivate our design. Averaging within and across layers gives every configuration a 64-number summary and the same 192-input comparison head (Appendix A). The shared-weight control repeats the same block, retaining the same parameter count at three and twelve rounds.

A variable-binding fitting control. We also built 32 disjoint training pairs whose negatives swap two variables, as in the test of Section 5. With the unchanged positional recipe, 3/3 source, 0/3 historical-EML, and 3/3 bounded-EML runs fit (Table 3). Full-feature refinement distinguishes every control group and all 300 swap-test groups for each representation. The historical failure is therefore not that measured input obstruction; its cause remains open.

Graph size alone does not explain fitting (Table 3). Bounded graphs are larger than historical graphs in the main population, yet their models fit all three small variable-swap controls while historical models fit none. Success depends on the representation, example construction, and learner together. Differing paths, sharing, and label distributions prevent isolating one structural cause of historical EML's failure.

Table 3: Fitting controls. Each entry counts passes among all tested configurations or runs; a dash means the representation was not part of that study. Every pass requires both training accuracy $\geq 95\%$ and BCE ≤ 0.2. The first two rows use a 32-pair task with template negatives; the third uses 32 pairs with variable-swap negatives. Their different examples prevent treating the rows as one controlled ablation.

Control	Source	Historical EML	Bounded EML
Six training and graph-summary choices, without whole-graph positions	5/6	0/6	–
Positions; 3/12 rounds, shared/separate weights, two seeds	8/8	8/8	–
Positions; 32 variable-swap pairs, three seeds	3/3	0/3	3/3

The position cap affects the representations differently. The common cap of 63 clips leaf-height features for 37.9% of bounded-EML nodes, 2.0% of historical-EML nodes, and no source nodes; it also clips root-distance features for 14.0% of bounded-EML nodes. Equal caps give unequal positional resolution. On the original swap panel, the encoded graphs remain distinguishable in the refinement check, but lost position detail may still affect learning. The comparison is therefore specific to this encoding configuration. Section 6 tests clipping at matched table size; direct cross-expression communication remains untested.

5 Original study: fitting is not generalization

The original controlled study uses the same 480 training pairs and 120 validation pairs for all three graph representations, with three random seeds. Table 1 separates it from the other studies. Training uses commutativity, distributivity, and regrouping with template-specific negatives. In commutativity and distributivity, these negatives compute the variable-swapped function but use different syntax; the swap construction preserves the positive expression's shape. Testing also includes binomial squares, differences of squares, and cubic expansion, with familiar and unfamiliar literals. All operators are present during training. These are six generator families, not a broad sample of mathematics.

Each representation uses twelve layers of width 64 (457,741 parameters). We chose this recipe after both source and historical EML passed the tiny development checks. Training uses FP32 and AdamW [28], with shared limits of 3,000 updates, 200 epochs or 1,800 active seconds. Validation loss selects a state; Appendix A gives optimizer and selection details.

Two tests ask different questions. Each test contains 600 pairs: 25 matched positive/negative groups in each of six template families and two literal groups. The tests share no source expressions, polynomial functions, or contexts with each other or the earlier populations. The first retains the original template-specific negatives. Its opposite-label examples match signed literal counts and nineteen source-structure counts. For difference of squares, the template negative is itself the same variable-swapped expression; that family has no construction contrast between the tests.

The second test swaps two variable names throughout the positive right-hand expression to create a negative. This keeps the measured, unlabeled position histograms (counts at each position) equal. A simple histogram classifier's development-validation performance motivated the test before either test's neural predictions were read. Matching removes that measured cue, not every possible shortcut. It also changes negative construction, so the score gap is not a pure causal estimate of reliance on shape.

Historical EML retains its original domain limitation: root agreement permits nonfinite intermediates, while strict finite-intermediate checks remain inconclusive. Its learning results therefore concern compiled syntax paired with source-equivalence labels; they are not validated recognition of equivalent finite-real EML programs.

A measured shape cue. A regularized linear classifier (ridge) using only node kind and the two clipped positions scores 50.0/59.2/65.8% on development validation for source/historical/bounded graphs (Appendix Table 7). Its vocabulary and scaling use training inputs only; neither test selects it. The swap test matches these histograms, guaranteeing this baseline's 50% accuracy. Neural models

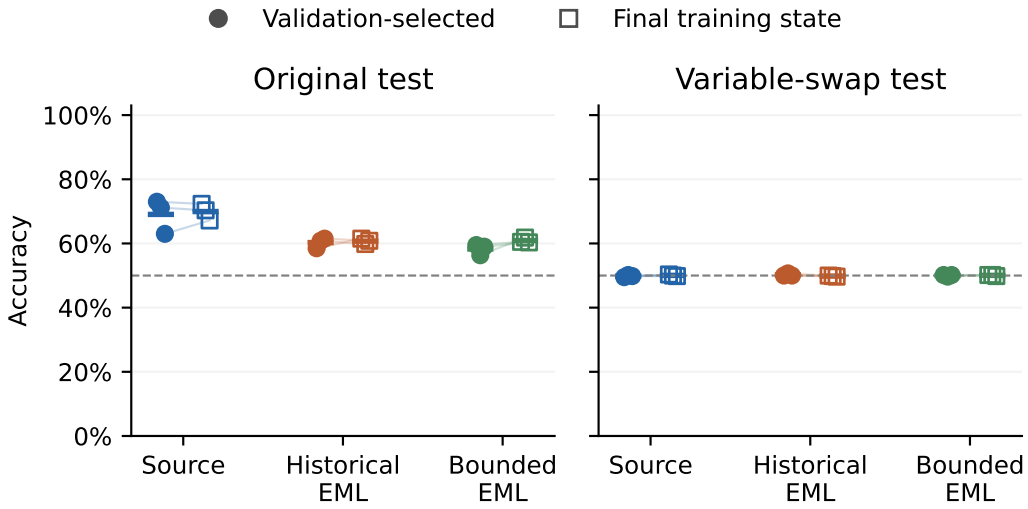


Figure 3: Original-population models: checkpoint choice and test population. Each point is one of three training seeds; short horizontal marks show means. Selected and final states come from the same three training runs. The dashed line is 50% accuracy. These are seed observations, not group-bootstrap confidence intervals. The final-state comparison is post hoc. Training uses template-specific negatives; the swap test preserves the positive right-hand side's syntax while changing variable identities. The tests use different unequal-example constructions and different positive contexts, so their accuracy gap does not isolate the causal effect of shape shortcuts.

also receive variable identities and ordered edges; source graphs additionally retain numerical literal features. Their test performance is empirical.

Training fit and model selection. Our primary, validation-selected states occur at updates 375–1,275. They fit the training data in only two source runs and no EML runs; by the final update all nine runs fit (Table 4). Final states are a diagnostic added after the original test results were read; no test information chooses a state. EML validation accuracy is 60–72% across both states, versus 73–97% for source graphs (Appendix Table 8). Thus fitting does not imply generalization even within the training distribution.

Table 4: Original-population training fit at the two saved states. S: validation-selected; F: final. Ranges cover all three seeds. A pass requires training accuracy at least 95% and BCE at most 0.2. Appendix C gives every seed's accuracy, BCE, update, and validation result.

Input	State	Train acc.% range	Train BCE range	Passes
Source	S	81.2–100.0	0.00861–0.349	2/3
Source	F	100.0	$(1.42\text{–}2.88) \times 10^{-5}$	3/3
Historical EML	S	77.7–86.2	0.306–0.447	0/3
Historical EML	F	96.9–98.1	0.0387–0.0683	3/3
Bounded EML	S	69.6–82.1	0.397–0.531	0/3
Bounded EML	F	98.8–99.8	0.00691–0.0385	3/3

For the selected models, original-test accuracy is $69.1 \pm 5.3\%$ for source graphs, $60.3 \pm 1.6\%$ for historical EML, and $58.3 \pm 1.7\%$ for bounded EML (mean and sample standard deviation over three seeds). Variable-swap accuracy is $49.8 \pm 0.3\%$, $50.2 \pm 0.4\%$, and $50.0 \pm 0.3\%$, respectively. The shared failure does not identify an EML-specific cause. Above-chance original-test means come from the familiar templates: all three unfamiliar templates score 45–52% for every representation and saved-state choice. On familiar commutativity, selected-state means are 96.3/93.7/87.7% on the original test and 50.3/50.0/50.3% on the swap test. Final states, all of which meet the fitting criterion, score 49.7–50.3% on the swap test (Figure 3).

Seed standard deviations from three runs are imprecise and cover only part of benchmark variability [29]. Supplementary group-bootstrap intervals condition on the fitted models and omit retraining variation. The public supplement supplies seeds, results, graph inputs, models, frozen recipes and saved-prediction replay (Appendices C and F).

Confidence fails as well as classification. Confidence need not track correctness [30], especially under distribution shift [31]. AUROC (area under the receiver operating characteristic curve) measures how often a randomly chosen positive scores above a randomly chosen negative, with half credit for ties; 0.5 is chance. Variable-swap AUROCs are approximately 0.5 for selected and final models, so changing one classification threshold cannot recover useful overall ordering. Mean swap-test BCE is 4.79, 1.18, and 1.14 for selected source, historical-EML, and bounded-EML models, respectively; final-state means are 6.73, 3.47, and 6.22. All exceed the balanced constant-prediction loss of 0.693. The share of all swap examples misclassified with at least 99% confidence rises from 30.6/4.0/4.4% (selected) to 41.3/30.3/36.9% (final). Continued fitting can thus increase confidence in incorrect predictions under this shift. These are measurements on the frozen tests, without threshold fitting or post hoc recalibration.

Replay with an exact checker. A separate rational-polynomial checker agrees with all 1,200 source-pair labels without consulting the graph compiler or stored labels. Ranking two candidate rewrites by saved scores saves checker calls on the original test; every swap-test conditional interval includes random ordering's 1.50 calls (Appendix Table 17). This post hoc replay restates within-group ranking (calls $= 2 -$ ranking); it does not measure wall-time or proof-search benefit.

6 Follow-up interventions

After the original results, we fixed each follow-up study's data, recipe, primary saved-state rule and analysis before training. No choices used fresh-test predictions.

Changing the binding-sensitive examples. To test distribution dependence, we replaced half the negatives in the original development population: 120 training and 30 validation pairs. Positives, contexts, splits, architecture and update budget stay fixed. Nine fresh fits use the same three seeds. Two new 600-pair panels, frozen before training and prediction, share all 300 positive contexts and differ in their ordinary/swap negatives. Original selected models are frozen; intervention models use the same selection rule on the modified validation examples. This changes training and model selection together. Familiar swaps use the intervention's training construction on fresh contexts from familiar templates; unfamiliar templates introduce a further distribution shift. For commutativity and distributivity, the original negatives already compute the swapped function with different syntax, so the intervention removes a syntactic cue rather than introducing entirely new negative functions.

Source-graph swap accuracy rises from $50.1 \pm 0.3\%$ to $70.9 \pm 2.0\%$ (Appendix Table 15). The paired increase is 20.9 percentage points from unrounded means, with a 95% group-bootstrap interval of [18.4,23.3] conditional on the three seeds; each seed improves. Familiar-template accuracy rises from 50.0% to $91.9 \pm 3.9\%$ (Table 5), while unfamiliar-template accuracy remains 50.0%. The latter conceals confident errors: selected source models classify 99.3–100% of unfamiliar-template pairs as negatives, with mean BCE 13.9, versus 4.2 originally. Final states, which involve no validation choice, show the same familiar-template gain ($90.8 \pm 3.9\%$; Appendix Table 15), so improvement is not merely a selection effect. Ordinary-negative accuracy changes much less, from 68.6% to 69.9%. The source-graph failure therefore depends on the training population; the improvement does not extend to unfamiliar templates.

All three source runs pass the fitting criterion at selected and final states. No EML intervention state passes, echoing historical EML's failure on the 32-pair swap control (Table 3). Selected EML states reach only 63.1–74.2% training accuracy, with BCE 0.482–0.657. At 3,000 updates, historical-EML training accuracy is 82.1–83.1% with BCE 0.263–0.287; bounded EML reaches 80.0–87.9% with BCE 0.245–0.382. Because these models also fail on their training examples, their test scores cannot isolate failure to generalize. Final ordinary-panel accuracy also falls in every seed, by 3.4 percentage points for historical EML and 4.0 for bounded EML. Twelve-round refinement finds no equal summaries within historical-EML training groups and only four bounded groups with equal summaries, giving an ideal accuracy ceiling of 99.17%; these obstructions cannot explain the much

Table 5: Fresh-panel accuracy (%, seed mean ± sample SD) before and after the binding-sensitive population intervention. Original selected models are frozen; intervention models select on modified validation examples. Each panel has 600 pairs with shared positive contexts; familiar/unfamiliar subsets each have 300 pairs. Source graphs improve on familiar swaps; final states show the same pattern (Appendix Table 15).

	Ordinary		Swap, familiar		Swap, unfamiliar	
Input	Orig.	Int.	Orig.	Int.	Orig.	Int.
Source	68.6 ± 6.0	69.9 ± 2.1	50.0 ± 0.0	91.9 ± 3.9	50.1 ± 0.5	50.0 ± 0.0
Historical EML	59.3 ± 1.5	57.5 ± 1.5	49.9 ± 0.5	49.6 ± 0.8	50.1 ± 0.4	50.0 ± 0.3
Bounded EML	57.4 ± 1.5	57.4 ± 1.4	49.8 ± 0.4	49.8 ± 0.4	50.0 ± 0.0	49.9 ± 0.2

Table 6: Matched-table-size position-cap control on bounded EML. Accuracy is seed mean ± sample SD (%). S: validation-selected; F: final. The paired cap-255 minus cap-63 difference is in percentage points, with a 95% group-bootstrap interval conditional on the three fitted seeds; it excludes retraining uncertainty. Both arms use the same 256-row position tables and 482,317 parameters.

Test	State	Cap 63	Cap 255	Paired difference
Original	S	58.2 ± 0.5	58.0 ± 0.4	$-0.17\ [-2.06, +1.83]$
Original	F	60.9 ± 1.8	60.4 ± 1.4	$-0.50\ [-2.22, +1.28]$
Swap	S	49.8 ± 0.6	50.3 ± 0.3	$+0.50\ [+0.05, +1.06]$
Swap	F	49.9 ± 0.4	49.9 ± 0.3	$-0.06\ [-0.56, +0.44]$

larger deficit. Many fresh-panel EML rankings are numerical ties (Appendix C). This intervention exposes a limitation of the tested EML recipe, without identifying its full cause or establishing that other EML learners would fail.

Removing position clipping. Six bounded-EML fits compare caps 63 and 255 on the original population, with three seeds and identical 256-row position tables, initialization and budget. The original baseline has 64 rows. Cap 255 was fixed before training using unlabeled development/test positions (maximum 253); test-only positions can address untrained rows. All six final states fit; no selected state does (Appendix Table 13). Swap accuracy remains near chance (Table 6); the selected-state half-point gain stays in that range although its conditional interval excludes zero. Original-test ranking falls by 4.3/5.5 percentage points (selected/final), increasing checker calls (Appendix Table 14). Selected updates differ; final updates match. Untrained rows do not establish the cause of the drop; other encodings and learners remain open.

7 Implications and limits

One operation increased graph size under the tested compiler. All nine original final models fit training but remain near chance on unfamiliar templates and variable swaps. Changing training examples helps source graphs on fresh swaps within familiar templates; EML models still underfit, and source errors on unfamiliar templates become more confident. More position detail does not resolve the bounded-EML swap failure in the tested control.

We test two related 480-pair polynomial training populations, three seeds and one graph-model family that processes each expression separately. Cross-graph comparison architectures [32, 33], broader renaming and training-size scaling remain untested. The EML compilers differ in graph structure and position information as well as domain behavior, preventing an isolated semantic-validity comparison. Numerical sampling certifies neither implementation. EML underfitting leaves other optimization recipes and architectures open.

For verifiable coding, training fit, compiled behavior and variable binding need separate checks. Accepting incorrect predictions can introduce software defects. Our checker-call replay does not measure end-to-end benefit. Neural equivalence scores remain suggestions for an independent checker, not proofs.

Acknowledgments and Disclosure of Funding

This research received no external funding.

LLM assistance. Large language models were used for manuscript refining, figure generation, and as code assistants. Scripts checked the claims against the saved result files and file integrity records used for the final analysis. No LLM is an evaluated model component or source of data or labels, and saved numerical outputs and validation scripts were used to check the reported results. The authors remain responsible for the core manuscript writing, scientific claims, analysis, accuracy, and integrity of the submission.

References

[1] Miltiadis Allamanis, Pankajan Chanthirasegaran, Pushmeet Kohli, and Charles Sutton. Learning continuous semantic representations of symbolic expressions. In *International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 80–88, 2017. URL https://proceedings.mlr.press/v70/allamanis17a.html.

[2] Andrzej Odrzywołek. All elementary functions from a single binary operator, 2026. URL https://arxiv.org/abs/2603.21852v2. arXiv:2603.21852v2.

[3] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. Equality saturation: A new approach to optimization. In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2009. URL https://www.cs.cornell.edu/~ross/publications/eqsat/.

[4] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL), 2021. doi: 10.1145/3434304.

[5] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020. URL https://arxiv.org/abs/2009.03393. arXiv:2009.03393.

[6] Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurélien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2205.11491.

[7] Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 2873–2882, 2018. URL https://proceedings.mlr.press/v80/lake18a.html.

[8] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.

[9] Guillaume Lample and François Charton. Deep learning for symbolic mathematics. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/1912.01412.

[10] David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1904.01557.

[11] Andrew Trask, Felix Hill, Scott E. Reed, Jack Rae, Chris Dyer, and Phil Blunsom. Neural arithmetic logic units. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/0e64a7b00c83e3d22ce6b3acf2c582b6-Abstract.html.

[12] Silviu-Marian Udrescu and Max Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6:eaay2631, 2020. doi: 10.1126/sciadv.aay2631.

[13] Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo. Neural symbolic regression that scales. In *International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 936–945, 2021. URL https://proceedings.mlr.press/v139/biggio21a.html.

[14] H. P. Barendregt, M. C. J. D. van Eekelen, J. R. W. Glauert, J. R. Kennaway, M. J. Plasmeijer, and M. R. Sleep. Term graph rewriting. In *PARLE Parallel Architectures and Languages Europe, Vol. 2: Parallel Languages*, volume 259 of *Lecture Notes in Computer Science*, pages 141–158, 1987. doi: 10.1007/3-540-17945-3_8.

[15] Jean-Christophe Filliâtre and Sylvain Conchon. Type-safe modular hash-consing. In *Proceedings of the 2006 ACM SIGPLAN Workshop on ML*, 2006. URL https://gallium.inria.fr/ml2006/accepted/5.html.

[16] Andrzej Odrzywołek. EML compiler v4, 2026. URL https://github.com/VA00/SymbolicRegressionPackage/blob/b3da148261199b46247306dfd92068f589778260/EML_toolkit/EmL_compiler/eml_compiler_v4.py. SymbolicRegressionPackage, formula reference at commit b3da1482.

[17] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. Society for Industrial and Applied Mathematics, 2009. doi: 10.1137/1.9780898717716.

[18] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027.

[19] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. doi: 10.1016/0005-1098(78)90005-5.

[20] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1263–1272, 2017. URL https://proceedings.mlr.press/v70/gilmer17a.html.

[21] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=ryGs6iA5Km.

[22] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 2019. URL https://arxiv.org/abs/1810.02244.

[23] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/f1c1592588411002af340cbaedd6fc33-Abstract.html.

[24] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=wTTjnvGphYj.

[25] Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/5d4834a159f1547b267a05a4e2b7cf5e-Abstract-Conference.html.

[26] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 5453–5462, 2018. URL https://proceedings.mlr.press/v80/xu18c.html.

[27] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 1725–1735, 2020. URL https://proceedings.mlr.press/v119/chen20v.html.

[28] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[29] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gael Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.

[30] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[31] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[32] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. Graph matching networks for learning the similarity of graph structured objects. In *International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pages 3835–3845, 2019. URL https://proceedings.mlr.press/v97/li19d.html.

[33] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. SimGNN: A neural network approach to fast graph similarity computation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*, 2019. URL https://arxiv.org/abs/1808.05689.

[34] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: Symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017. doi: 10.7717/peerj-cs.103.

[35] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1801.07606.

[36] Kenta Oono and Taiji Suzuki. Graph neural networks exponentially lose expressive power for node classification. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/1905.10947.

[37] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2006.05205.

[38] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2111.14522.

[39] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio, and Michael M. Bronstein. On over-squashing in message passing neural networks: The impact of width, depth, and topology. In *International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 7865–7885, 2023. URL https://proceedings.mlr.press/v202/di-giovanni23a.html.

[40] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL https://arxiv.org/abs/1607.06450. arXiv:1607.06450.

[41] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1979. doi: 10.1214/aos/1176344552.

A Task, inputs, and model specification

Task and populations. Each example is a pair of expressions labeled positive (equivalent) or negative (non-equivalent). Every matched group contains one of each with the same left expression. Exact rational-polynomial expansion supplies the labels; the symbolic pipeline uses SymPy [34]. The six template families are commutativity, distributivity, regrouping, binomial square, difference of squares, and cubic expansion. Only the first three appear in the 480-pair training population; 120 separate pairs are used for validation. The original two test sets each contain 300 groups, or 600 pairs. Within each set there are 25 groups per template and literal group. Familiar literals are checked against numbers actually present in training, including derived coefficients. An unfamiliar-template cell can also change algebraic structure; these axes do not isolate every possible source of difficulty.

Input encoding. Each node has an operation/variable label, node kind, root flag and root order. Source-graph numerical constants have a category, sign, the two values $\log(1+|p|)$ and $\log(1+|q|)$ for the exact rational p/q, and a bounded categorical bucket. Previously unseen source constants therefore retain numerical features even when their categorical identifier is unfamiliar. EML graphs instead contain only the primitive, variables and the unit constant: their node-value fields follow fixed operator/leaf templates, while source coefficient magnitudes are encoded by graph structure. They do not receive the source coefficients as additional scalar inputs. A bucket alone is not an injective encoding. All actual input fields are used in the refinement checks.

Each logical parent-to-child edge produces a forward and a reverse message with direction, ordered child slot, and role labels. Repeated references remain repeated messages. Vocabulary capacities are kind 16, label 256, value category 8, value bucket 1024, sign 3, root flag 2, root order 8, direction 2, child slot 8, and role 4. The vocabulary is fitted on training inputs and reused unchanged.

Encoder and pair head. Graph depth raises distinct concerns in prior work: repeated propagation can make node states increasingly alike [35, 36], while fixed-width states can compress information arriving from distant nodes [37–39]. These results motivate diagnostics, but neither mechanism is established as the cause of our EML fitting failure. The cited analyses concern their stated model assumptions; they are not proofs about this implementation.

Embeddings are learned vectors for categorical values. Categorical node embeddings and a linear projection of the two numerical features sum to a width-64 initial state h_v^0. The positional variant also sums two learned 64-entry embeddings: shortest root distance and longest leaf height, clipped at 63. Direction, slot, and role embeddings sum to an edge vector a_{uv}. ReLU sets negative values to zero; MLP denotes a multilayer feed-forward network, and LN denotes layer normalization. In layer ℓ,

$$m_v = \sum_{u \to v} \mathrm{ReLU}(h_u + W_\ell a_{uv}), \qquad u_v = \mathrm{LN}\left(h_v + \mathrm{MLP}_\ell((1+\epsilon_\ell)h_v + m_v)\right).$$

The learned scalar ϵ_ℓ starts at zero. The message MLP has widths 64–64–64 and a ReLU. A second residual block uses a feed-forward network (FFN):

$$\widetilde{h}_v = \mathrm{LN}\left(u_v + \mathrm{FFN}_\ell(u_v)\right), \qquad h_v^{\ell+1} = 0.8\widetilde{h}_v + 0.2h_v^0,$$

where the FFN has widths 64–128–64 with a ReLU. Layer normalization [40] operates on each node's feature vector. Dropout is zero. Twelve separate blocks are used in the main study. The graph summary is the mean of the twelve per-layer node means. A shared encoder processes the two expressions; their summaries a, b produce $[a+b, |a-b|, a \odot b]$. A 192–64–1 ReLU network returns a logit, a score converted to an equivalence probability. This construction is symmetric under swapping expressions. It does not directly align their nodes.

Optimization and saved models. AdamW [28] uses learning rate 10^{-3} and zero weight decay. Gradients are clipped to norm 1. Batches contain 16 complete two-label groups. Group order is deterministically shuffled each epoch; no group is divided between training and validation. The main fits use at most 3,000 updates, 200 epochs, or 1,800 active seconds, with validation every five epochs. Minimum validation BCE selects the primary model. Final-state inference uses the last already-saved training update and was specified after the original test results were known; it is a post hoc diagnostic. It does not replace the primary selected-model results. Random seeds and continuous training outcomes are listed below.

Metrics and uncertainty. Classification predicts equivalence when logit > 0. Within each matched group, ranking is 1 if the positive pair's logit exceeds the negative pair's by more than 10^{-6}, 0 if it is lower by more than that amount, and $1/2$ otherwise. Macro-F1 averages the two class-specific F1 scores, which balance precision (how often a predicted class is correct) and recall (how many examples of that class are found). AUROC ranks positives against negatives across the full test, with 0.5 indicating chance. BCE is the mean binary cross-entropy prediction loss on all pairs; lower is better, and balanced constant guessing gives approximately 0.693. Results include all seeds. The sample standard deviation describes observed seed variation; paired 2,000-draw bootstrap intervals [41] resample complete groups jointly across representations and are conditional on the three fitted seeds. Template/literal aggregates weight pairs equally; each original test cell has the same number of pairs.

Small variable-swap fitting control. The binding-development population uses the same variable-swap negative construction, with 32 balanced familiar-template training pairs and 32 validation pairs. Exact source strings, polynomial functions, and generator contexts are disjoint from the earlier populations. The recipe is fixed before fitting. Each tiny fit receives at most 1,500 updates or 900 active seconds and is checked every 50 updates; accuracy $\geq .95$ and BCE $\leq .2$ must hold for three consecutive checks. The completed tiny controls assess fitting only; they do not support a new claim about generalization.

Binding-sensitive population intervention. The larger intervention retains the original 480 training and 120 validation pairs, positive examples, contexts and split membership. It replaces half the negatives in each split: 120 training and 30 validation pairs. A fixed hash ranking chooses eligible contexts proportionally across the three familiar families. Each replacement globally swaps the first two variables in the positive right expression; exact rational algebra and an unequal-coordinate counterexample verify its negative label. Its literal counts, ordered slots and unlabeled position histograms match the positive. Nine fresh fits reuse the original seeds, initialization procedure, batch order, architecture and update budget. The validation-loss rule is unchanged, but its examples change; the original selected states are frozen rather than reselected. This is a population intervention involving training and selection, not an isolated change to training alone.

Two fresh 600-pair panels share all 300 positive contexts and differ in negative construction. They exclude source strings, polynomial functions and contexts in the earlier populations. The generator uses seed 2026100911 and the same six-family/two-literal-group quotas. In commutativity and distributivity, the original template negatives already compute the swapped function with a different syntax; the intervention removes that syntax difference. Regrouping introduces new negative functions. All 50 difference-of-squares negatives are identical across the fresh panels. Selected states are primary and final states secondary, as fixed before these fits; all model states and seeds are reported. Familiar versus unfamiliar templates also differ in graph geometry and cannot isolate identity familiarity alone.

B Finite-real construction and numerical checks

Statement and assumptions. Let an accepted source expression be generated by exact Integer/Rational leaves, real variables, and ordered Add/Mul. Each variable has a finite closed interval with rational endpoints compatible with its declared assumptions. The compiler returns a DAG containing only 1, variables, and $E(a, b) = \exp(a) - \log b$. In exact real arithmetic, for every assignment in the box, its root equals the source value, every primitive intermediate is finite real, and every primitive logarithm has a strictly positive operand. This statement depends on the elementary identities below, exact interval arithmetic, parser correctness, and the implementation of sharing. It is a compositional mathematical argument with executable checks, not a proof-assistant certificate.

Primitive domain lemmas. For finite real t, $X(t) = E(t, 1) = \exp(t)$ is strictly positive and finite. For strictly positive finite t,

$$L(t) = E(1, X(E(1, t))) = e - \log\big(\exp(e - \log t)\big) = \log t.$$

Every logarithm in this composition has a positive operand. If $a > 0$ and b is any finite real, then

$$S(a, b) = E(L(a), X(b)) = a - b.$$

Its first-operand positivity side condition is essential. With $e = X(1)$ and $e - 1 = S(e, 1)$, define

$$N(t) = S(1, S(e, S(e - 1, t))) = -t.$$

The three first operands are the positive constants $1, e, e - 1$; intermediate subtraction results may have either sign without violating these conditions. Rational bounds $2 < e < 3$ enclose the helper constants. Zero is $L(1)$. Positive integers are built by doubling/addition from one; negative integers use N.

Safe addition. If the lower bound of a is positive, define $A(a, b) = S(a, N(b))$. For integer $K \geq 0$,

$$T_K(t) = S(1 + K, S(1, t)) = t + K; \qquad T_{-K}(t) = S(1, S(1 + K, t)) = t - K.$$

These translations require only positive constant first operands. Otherwise, if $l \leq a$ is a rational lower bound, choose $K = 1 - \lfloor l \rfloor$, giving $a + K \geq 1$, and define

$$A(a, b) = T_{-K}\big(S(T_K(a), N(b))\big) = a + b.$$

The enclosing output interval is $[a_l + b_l, a_u + b_u]$. Source variables may share dependencies; interval overapproximation remains safe [17].

Products and rational constants. For positive operands, $P(a, b) = X(A(L(a), L(b))) = ab$. Rational bounds for logarithms follow from $1 - 1/x \leq \log x \leq x - 1$ for $x > 0$. Safe addition handles a negative or zero logarithm. A strictly negative operand is negated, the product is computed recursively, and the result is negated. In the remaining zero-containing or sign-crossing cases choose positive integers K, L with $a + K \geq 1$ and $b + L \geq 1$, and use

$$ab = P(a + K, b + L) - P(a + K, L) - P(K, b + L) + KL.$$

Subtraction uses A and N. The resulting interval is the minimum/maximum of the four endpoint products. For integer $q > 0$, $X(N(L(q))) = 1/q$; multiplication with the signed numerator constructs a rational without adding numerical leaves.

Induction and sharing. Leaves equal their exact literal values or satisfy their declared variable boxes. Each operation above preserves the source value, carries a sound rational interval, and satisfies every primitive domain condition. Structural induction establishes the statement for every accepted syntax tree. Sharing identical ordered nodes merges only equal subexpressions and preserves repeated child slots. Removing nodes not used by the output affects only unused helpers. Unsupported source syntax is rejected; fixed nonnegative integer powers must be expanded before compilation.

Numerical evidence and its limits. The replay traverses exported primitive nodes child before parent, checking logarithm positivity, intermediate finiteness, and an explicit exponent cap. It does not substitute the macro identities. Exact rational polynomial evaluation supplies source values. All 21,107 source endpoints compile; their largest new graph has 1,463 nodes. Three assignments (both box corners and one source-hashed coordinate assignment) at 80 and 120 digits yield $21{,}107 \times 3 \times 2 = 126{,}642$ agreeing whole-graph evaluations. Each graph evaluation checks every traversed primitive node; this total is not a count of individual primitive-node evaluations. Agreement uses scaled tolerance 10^{-20} and exponent cap 10^9. The full numerical ledger retains source/graph/record identifiers and outcome classes.

Finite real in exact arithmetic does not mean representable in standard hardware floating point. Nested exponentials can have enormous finite magnitudes, and subtraction can be ill-conditioned [18]. The numerical audit is finite sampling, not a universal quantified proof or an independently verified implementation. Conversely, the mathematical domain argument covers the whole declared box subject to its trust boundary, rather than only the sampled assignments. Historical EML graphs keep their original strict-inconclusive outcomes; new finite graphs constitute a different representation and can have different size, distances, and optimization behavior.

Checks sensitive to variable binding. Equal-coordinate box corners are inadequate for detecting swapped variables: $2x + y$ and $2y + x$ agree whenever $x = y$. We therefore evaluate the saved finite graphs at each structure-matched group's exact unequal-coordinate counterexample. At 80 and 120 decimal digits, we compare the left expression and both right expressions with exact rational values, check every visited primitive's domain, and check that the negative pair remains numerically distinct. Outcomes are retained separately as agreement, mismatch, and inconclusive. These checks improve implementation evidence for the tested inputs; they do not turn sampling into a proof.

What the simple shape baseline sees. The motivating baseline is a ridge classifier with penalty 0.001. It counts combinations of node kind, distance from the output, and height above the leaves; both distances are clipped at 63. It omits variable identities and numerical values. Its vocabulary and feature scaling use training data only. Table 7 shows above-chance validation accuracy for both EML inputs on this 120-pair population. Bounded EML's perfect training score and lower validation score also illustrate the gap between fitting these counts and generalizing from them. Source counts score 50% on both populations, so this measured cue cannot explain the source model's ordinary-test performance.

Table 7: Shape-only ridge baseline on the 480 training and 120 development-validation pairs. These are single deterministic baseline fits, not neural test results. Neither frozen 600-pair test was used to fit or select this baseline.

Representation	Training accuracy	Validation accuracy
Source	50.00%	50.00%
Historical EML	67.50%	59.17%
Bounded EML	100.00%	65.83%

The swap test equalizes these histograms within each group, so a classifier using only them scores exactly 50% by construction. The neural models also see variable identities and ordered edges; source graphs additionally retain numerical literal features. Their near-chance scores are an empirical result.

C Complete results

Table 4 in the main text summarizes the original selected and final training outcomes; Table 8 lists each seed. Template summaries follow. Every test seed, full precision, template/literal cells, prediction logits and grouped intervals are retained in the machine-readable supplement. A training pass describes a particular run, not a precise success probability for future training. The complete

Table 8: All main-study saved states. S: validation-selected; F: final. Seeds 21/22/23 mean 20260921/22/23. Accuracy is a percentage. The joint fitting criterion is accuracy at least 95% and BCE at most 0.2.

Input	Seed	State	Update	Train acc.	Train BCE	Val. acc.	Val. BCE
Source	21	S	1275	100.0	0.00861	94.2	0.217
Source	21	F	3000	100.0	1.47e-05	91.7	0.479
Source	22	S	975	100.0	0.015	96.7	0.0781
Source	22	F	3000	100.0	1.42e-05	94.2	0.39
Source	23	S	375	81.2	0.349	73.3	0.476
Source	23	F	3000	100.0	2.88e-05	82.5	1.15
Historical EML	21	S	375	77.7	0.447	66.7	0.556
Historical EML	21	F	3000	97.3	0.0614	69.2	1.06
Historical EML	22	S	525	84.0	0.33	69.2	0.569
Historical EML	22	F	3000	98.1	0.0387	65.8	1.1
Historical EML	23	S	525	86.2	0.306	70.8	0.539
Historical EML	23	F	3000	96.9	0.0683	69.2	1.01
Bounded EML	21	S	600	82.1	0.397	66.7	0.587
Bounded EML	21	F	3000	98.8	0.0349	67.5	2.02
Bounded EML	22	S	450	69.6	0.531	60.0	0.612
Bounded EML	22	F	3000	99.8	0.00691	68.3	1.97
Bounded EML	23	S	525	76.7	0.445	65.8	0.598
Bounded EML	23	F	3000	99.0	0.0385	71.7	2.57

per-group numerical outcomes and source/input hashes are in the supplementary ledger. The matched-table-size clipping comparison uses the original development and two test populations. All six runs reached 3,000 updates; active time was 1,190–1,475 seconds per fit, totaling 2.13 hours of cumulative per-fit wall-clock time, including periodic validation and a per-update throttle; test inference and time lost to interruptions are excluded. Both conditions use 256-row root-distance and leaf-height embeddings; the clipping cap is the sole configured difference. Positions beyond those seen in training address untrained rows in the cap-255 condition. Raw prediction logits, every

Table 9: Original test (template-specific negatives): accuracy in percent, selected / final, averaged equally over three seeds and 100 pairs per template. The first three templates occur in training; the last three do not.

Template	Source S/F	Historical S/F	Bounded S/F
commutativity	96.3 / 95.0	93.7 / 95.0	87.7 / 89.0
distributivity	77.3 / 88.0	54.0 / 58.0	49.3 / 59.0
regrouping	91.0 / 87.0	61.3 / 60.7	66.7 / 70.0
binomial square	50.3 / 50.7	50.7 / 49.3	50.7 / 49.3
difference squares	49.7 / 49.3	50.0 / 50.0	50.3 / 50.3
cubic	49.7 / 49.7	52.0 / 51.3	45.0 / 47.7

Table 10: Variable-swap test: accuracy in percent, selected / final, averaged equally over three seeds and 100 pairs per template. The first three templates occur in training; the last three do not.

Template	Source S/F	Historical S/F	Bounded S/F
commutativity	50.3 / 50.0	50.0 / 50.0	50.3 / 50.7
distributivity	50.0 / 50.0	50.0 / 50.0	49.7 / 49.3
regrouping	49.7 / 49.3	51.7 / 49.3	50.0 / 50.0
binomial square	50.0 / 50.0	49.7 / 49.7	50.0 / 50.0
difference squares	49.0 / 51.3	50.0 / 50.0	50.0 / 50.0
cubic	50.0 / 49.7	50.0 / 50.0	50.0 / 50.3

Table 11: Full-feature exact color refinement on 300 opposite-label graph pairs per representation. Twelve rounds and normalized color histograms account for mean pooling. Different colors do not guarantee that the fixed parameterization or optimizer learns a separator.

Input	Distinguishable	Indistinguishable	Unresolved
Source	300	0	0
Historical EML	300	0	0
Bounded EML	300	0	0

Table 12: Additional paired structure measurements on the same 2,700 source endpoints. Clipped percentages count nodes with true position greater than 63, weighted equally over all nodes in each representation. Finite-domain construction also changes graph size, depth, and how much position information is compressed.

Input	Mean nodes	Maximum depth	Root clipped%	Leaf clipped%
Source	11	6	0.0	0.0
Historical EML	111	95	0.0	2.0
Bounded EML	323	253	14.0	37.9

seed/test metric, and paired group-bootstrap contrasts are retained in the supplementary clipping analysis. These intervals condition on the three fitted seeds. Two clipping fits and one binding fit resumed from rolling checkpoints; consumed budgets and random states were restored rather than reset. All nine binding fits reached 3,000 updates, with recorded per-fit active time 233–1,558 seconds (1.83 hours in total, using the same retained-time convention).

The fresh-panel comparison scores every original/intervention selected/final state on both panels, including all underfitting states. Ordinary-panel source accuracy changes much less than swap accuracy, with selected-state differences negative in two seeds and positive in one; the supplement retains these contrasts alongside every family/literal cell. The original selected states are not reselected on the new validation data. For selected source intervention models, familiar-template swap accuracy is $91.1 \pm 5.6\%$ with familiar literals and $92.7 \pm 2.3\%$ with unfamiliar literals; unfamiliar-template accuracy is 50.0% in both cohorts. On unfamiliar swap templates, final source models classify every pair as negative, with mean BCE 16.5 versus 5.2 for the original final models. Final ordinary-panel accuracy decreases by 3.4 percentage points for historical EML (95% conditional interval

Table 13: Training fit for every clipping-control state. S: validation-selected; F: final. A pass requires accuracy $\geq 95\%$ and BCE ≤ 0.2. All six fits reach 3,000 updates; final states all pass and selected states all fail.

Cap	Seed	State	Training accuracy (%)	Training BCE
63	20260921	S	68.12	0.546044
63	20260921	F	99.58	0.020362
63	20260922	S	65.42	0.557395
63	20260922	F	98.75	0.033379
63	20260923	S	74.17	0.455776
63	20260923	F	99.79	0.012890
255	20260921	S	67.92	0.561442
255	20260921	F	100.00	0.000034
255	20260922	S	67.92	0.562786
255	20260922	F	100.00	0.000239
255	20260923	S	70.00	0.544400
255	20260923	F	100.00	0.002312

Table 14: Every paired position-cap contrast (255 minus 63), in percentage points. Seed columns abbreviate 20260921–23. S: selected, the primary state; F: final. Intervals resample groups conditional on these fitted seeds and do not include uncertainty from retraining. Ranking is within matched positive/negative groups.

Test, metric	State	Seed 21	Seed 22	Seed 23	Mean	Conditional 95% interval
Original, Accuracy	S	-0.83	-0.50	$+0.83$	-0.17	$[-2.06, +1.83]$
Original, Accuracy	F	$+3.00$	-3.00	-1.50	-0.50	$[-2.22, +1.28]$
Original, Ranking	S	-4.17	-5.17	-3.50	-4.28	$[-8.22, -0.33]$
Original, Ranking	F	-6.83	-5.67	-4.00	-5.50	$[-9.33, -1.78]$
Swap, Accuracy	S	$+0.83$	$+0.33$	$+0.33$	$+0.50$	$[+0.05, +1.06]$
Swap, Accuracy	F	-0.17	$+0.17$	-0.17	-0.06	$[-0.56, +0.44]$
Swap, Ranking	S	$+2.33$	-3.00	$+2.00$	$+0.44$	$[-3.67, +4.67]$
Swap, Ranking	F	-8.83	$+4.17$	$+0.00$	-1.56	$[-6.17, +3.00]$

$[-5.5, -1.6]$) and 4.0 for bounded EML ($[-6.5, -1.6]$), with every seed negative. These intervals condition on the fitted seeds and do not identify the cause of underfitting.

Near-tied scores. On the fresh swap panel, 14.0–49.0% of historical-EML groups and 2.0–7.3% of bounded-EML groups are ties at the 10^{-6} ranking rule, across original/intervention selected/final states and seeds. Source models have no such ties. Observed batch-composition and CPU/GPU logit differences can exceed this threshold; near-chance EML ranking can therefore be numerically fragile rather than a stable random ordering. Published ranking uses the saved GPU scores and the stated tie rule. The hardware replay below reports the observed changes without assigning a unique numerical cause.

Fresh binding-panel input checks. For the new population intervention, the same full-feature, 12-round refinement check distinguishes all 300 source and historical-EML swap groups, but 298/300 bounded-EML groups. The fresh bounded graphs reach height 293, beyond the original clipping control's maximum of 253; that control therefore does not establish clipping's contribution on this new population. The two unresolved groups are familiar templates (commutativity and distributivity) with unfamiliar literals; this is a limit of the stated refinement check, not evidence of identical serialized graphs. A uniform audit separately covers all six original/intervention development and test populations. For each of their 300 groups, it selects the first exact counterexample with unequal active-variable coordinates on the grid $\{1/4, 1/2, 1, 2, 4\}$, without consulting model scores. All 10,800 saved bounded-graph evaluations agree: six populations, 300 groups, three endpoint roles and two precisions (80/120 digits). All 3,600 precision-specific negative distinctions are preserved. Every visited primitive must remain finite and real with a positive logarithm argument; root error must be at most 10^{-20} relative to $1 + |y|$. The ledger retains every outcome without filtering. These finite checks neither certify the compiler nor eliminate architectural limitations. The supplementary input-controls records include every outcome and the panel-name mapping.

Table 15: Fresh swap-panel results, mean ± sample SD (%) over three seeds. O/I: original/intervention population; S/F: selected/final saved state. Familiar and unfamiliar subsets each contain 300 pairs. Ranking compares the positive and unequal member of each group.

Input	Model	All accuracy	Familiar	Unfamiliar	All ranking
Source	O/S	50.1 ± 0.3	50.0 ± 0.0	50.1 ± 0.5	51.1 ± 1.3
Source	I/S	70.9 ± 2.0	91.9 ± 3.9	50.0 ± 0.0	76.0 ± 4.4
Historical EML	O/S	50.0 ± 0.3	49.9 ± 0.5	50.1 ± 0.4	48.2 ± 3.1
Historical EML	I/S	49.8 ± 0.5	49.6 ± 0.8	50.0 ± 0.3	48.1 ± 2.0
Bounded EML	O/S	49.9 ± 0.2	49.8 ± 0.4	50.0 ± 0.0	50.7 ± 1.2
Bounded EML	I/S	49.8 ± 0.2	49.8 ± 0.4	49.9 ± 0.2	46.7 ± 1.8
Source	O/F	49.9 ± 0.3	49.8 ± 0.2	50.1 ± 0.5	53.0 ± 2.2
Source	I/F	70.4 ± 1.9	90.8 ± 3.9	50.0 ± 0.0	75.9 ± 3.7
Historical EML	O/F	49.8 ± 0.2	49.6 ± 0.4	50.0 ± 0.0	50.8 ± 0.6
Historical EML	I/F	50.3 ± 0.8	50.7 ± 1.7	50.0 ± 0.0	53.2 ± 1.8
Bounded EML	O/F	49.8 ± 0.3	49.9 ± 0.2	49.8 ± 0.7	49.9 ± 3.5
Bounded EML	I/F	49.9 ± 0.6	49.9 ± 1.8	50.0 ± 0.6	46.1 ± 1.7

Table 16: Every binding-comparison seed and saved state. Training metrics use each model's own population. Test accuracy and ranking use the shared fresh swap panel. O/I and S/F follow Table 15.

Input	Model	Seed	Update	Train acc. (%)	Train BCE	Swap acc. (%)	Ranking (%)
Source	O/S	20260921	1275	100.00	0.00861	49.83	51.33
Source	O/F	20260921	3000	100.00	0.00001	50.33	53.33
Source	O/S	20260922	975	100.00	0.01498	50.00	52.33
Source	O/F	20260922	3000	100.00	0.00001	49.67	55.00
Source	O/S	20260923	375	81.25	0.34909	50.33	49.67
Source	O/F	20260923	3000	100.00	0.00003	49.83	50.67
Source	I/S	20260921	1575	100.00	0.00389	72.17	76.67
Source	I/F	20260921	3000	100.00	0.00003	70.67	75.00
Source	I/S	20260922	1425	100.00	0.00210	72.00	80.00
Source	I/F	20260922	3000	100.00	0.00002	72.17	80.00
Source	I/S	20260923	2175	99.17	0.02264	68.67	71.33
Source	I/F	20260923	3000	100.00	0.00003	68.33	72.67
Historical EML	O/S	20260921	375	77.71	0.44734	50.33	51.83
Historical EML	O/F	20260921	3000	97.29	0.06143	50.00	50.67
Historical EML	O/S	20260922	525	83.96	0.33028	49.83	46.17
Historical EML	O/F	20260922	3000	98.12	0.03869	49.67	51.50
Historical EML	O/S	20260923	525	86.25	0.30635	49.83	46.67
Historical EML	O/F	20260923	3000	96.88	0.06834	49.67	50.33
Historical EML	I/S	20260921	375	64.38	0.65690	50.17	49.83
Historical EML	I/F	20260921	3000	82.92	0.27136	51.17	55.17
Historical EML	I/S	20260922	750	74.17	0.48161	50.00	45.83
Historical EML	I/F	20260922	3000	82.08	0.28667	49.50	52.83
Historical EML	I/S	20260923	375	68.33	0.58690	49.17	48.50
Historical EML	I/F	20260923	3000	83.12	0.26303	50.33	51.67
Bounded EML	O/S	20260921	600	82.08	0.39670	50.00	52.00
Bounded EML	O/F	20260921	3000	98.75	0.03490	50.00	46.67
Bounded EML	O/S	20260922	450	69.58	0.53123	49.67	49.67
Bounded EML	O/F	20260922	3000	99.79	0.00691	49.50	53.67
Bounded EML	O/S	20260923	525	76.67	0.44533	50.00	50.33
Bounded EML	O/F	20260923	3000	98.96	0.03852	50.00	49.50
Bounded EML	I/S	20260921	450	65.00	0.61908	50.00	45.33
Bounded EML	I/F	20260921	3000	80.00	0.38217	49.67	44.33
Bounded EML	I/S	20260922	1050	69.38	0.55981	49.67	48.67
Bounded EML	I/F	20260922	3000	83.33	0.31383	49.50	46.17
Bounded EML	I/S	20260923	750	63.12	0.61411	49.83	46.00
Bounded EML	I/F	20260923	3000	87.92	0.24487	50.67	47.67

Training-set refinement check. The same check covers every original and intervention development group, with bit-identical encoded left inputs verified within each opposite-label pair. All original training/validation groups are distinguishable by twelve rounds in every representation. After the intervention, all 240 historical-EML training groups remain distinguishable, while four bounded-EML groups have equal twelve-round summaries. Their ideal mean-summary accuracy ceiling is 99.17%, above the 95% fitting criterion; they cannot account for the much larger observed underfitting. Three-round training obstructions increase from 9 to 80 historical-EML groups and from 0 to 24 bounded-EML groups, with none for source graphs. All 60 intervention validation groups are distinguishable at twelve rounds. Different refinement summaries permit separation without

guaranteeing it for a fixed-width model or a particular optimizer; floating-point execution can also depart from ideal-arithmetic symmetry.

Exact-checker replay. The checker ranks two candidate rewrites using the saved scores and accepts only an exact rational-polynomial match. The table retains all model/state combinations; the conditional group-bootstrap intervals for swap-test calls include the random-order expectation of 1.50 in every case.

Table 17: Post hoc two-candidate checker replay and confidence under variable swaps. Calls are mean (sample SD) over three seeds and 300 groups per test; random ordering needs 1.50 calls in expectation. S/F denote selected/final states. "Wrong, $\geq 99\%$" is the percentage of all swap examples that are misclassified with at least 99% confidence.

Input	State	Original calls	Swap calls	Swap BCE	Wrong, $\geq 99\%$
Source	S	1.308 (0.066)	1.486 (0.037)	4.79	30.6%
Source	F	1.259 (0.015)	1.498 (0.028)	6.73	41.3%
Historical EML	S	1.388 (0.022)	1.474 (0.031)	1.18	4.0%
Historical EML	F	1.379 (0.022)	1.494 (0.041)	3.47	30.3%
Bounded EML	S	1.391 (0.013)	1.501 (0.025)	1.14	4.4%
Bounded EML	F	1.374 (0.017)	1.496 (0.050)	6.22	36.9%

D Supporting diagnostics and earlier experiments

Larger historical learning population. An earlier study trained three-layer, width-64 source and historical-EML models (301,956 parameters each) on 50,000 pairs, with 5,000 each for validation, independent and identically distributed (IID) testing, and distribution-stress testing. Mean IID macro-F1 over three seeds was 0.9971 and 0.9802. On 4,492 numerically agreeing IID pairs it was 0.9970 and 0.9793. A post-training audit of 22,271 endpoints recorded 18,331 numerical agreements, 2,388 compiled-inconclusive outcomes, 1,551 source-inconclusive outcomes and one mismatch. Filtering test inputs does not repair uncertain training inputs; strict finite-intermediate checks on the later 21,107 retained endpoints remain inconclusive. Different data, architecture, grammar and negatives prevent treating this study as a training-size comparison or evidence of variable-binding transfer.

Limits of earlier comparisons. The first four-model grid failed to fit even source graphs and introduced an unseen power label. A later comparison expanded powers into multiplication, but concatenating layer outputs changed the comparison-head size with depth. Those results cannot isolate the effect of propagation depth. The archived records retain the original outcomes; the fixed-size and shared-weight controls in the main text address specific limitations of those earlier comparisons.

The symmetric six-choice search gives five source-graph passes and zero historical-EML passes; the subsequent 16 position controls all pass. The first failed grid and corrected concatenated-readout grid remain separate in the archived result records. For the initial historical-EML model with a fixed-width summary, the FP32 median matched distance is 5.82e-07; reversing message order for the same graph gives median distance 5.32e-07. This is one measured numerical perturbation, not a complete model of numerical error.

Table 18: Higher-precision replay of identical fixed-readout weights. Relative separation divides matched distance by the mean norm of the two summaries. This removes a common scalar scale factor; it does not identify a unique training mechanism.

Input	State	Absolute separation	Relative separation
Source	initial	0.0249	0.00479
Source	trained	3.47	0.799
Historical EML	initial	2.25e-10	4.12e-11
Historical EML	trained	2.91e-15	5.31e-16

E Auditing representation costs

The supplement distinguishes three levels of evidence. A source-only ledger covers all 250,000 historical expressions; reparsing the locally retained sources reproduces the median 29, 95th percentile 123, and size range 2–213, with no parser failures. It has no compiled-size columns. The original per-expression compiled-cost ledger is unavailable, so the historical EML summaries below are transcribed records, with provenance and checked source literals. A separate 2,700-expression polynomial ledger recounts the serialized source, historical, and bounded graph inputs and agrees with all 8,100 saved graph measurements. It verifies this learning population, not the broader historical expansion distribution.

Table 19: Saved historical tree-expansion summaries by source family. Ratios divide EML tree nodes by source tree nodes for each expression. These are recorded aggregates, not a newly regenerated cost ledger.

Family	Expressions	Median ratio	90th percentile
Algebraic	70,000	25.58	33.34
Exponential/logarithmic	40,000	32.79	58.71
Powers/division/rational	40,000	44.16	67.41
Stress	25,000	45.74	70.41
Mixed elementary	35,000	133.93	987.50
Trigonometric/hyperbolic	40,000	270.46	4,566.15

Recorded median expansion by source-tree size is 110.14 for 1–8 nodes, 38.69 for 9–16, 33.88 for 17–32, 38.83 for 33–64, 46.87 for 65–128, and 50.45 above 128. Expansion is therefore not monotonic in source size under this generator. Historical overall 90th/95th/99th percentiles are 385.07/1,187.82/10,448.60, with maximum 6,481,679.31. No per-family 95th/99th percentiles or maxima are recoverable from these records.

Table 20: The 2,700 learning-study expressions: per-expression shared-DAG expansion relative to the source DAG. All endpoints are retained. The two rightmost columns distinguish aggregation conventions.

Compiler	Median	95th percentile	Maximum	Mean ratio	Ratio of totals
Historical EML	9.67	15.16	17.05	10.06	10.12
Bounded EML	24.08	59.16	160.67	28.91	29.40

The learning population's historical tree ratio has median 117.13 and maximum 46,037.39; its tail concentrates in cubic examples with unfamiliar literals. Unlike the 250,000-expression corpus, these inputs contain no transcendental families, which drive the historical tail. The ledger records family, literal cohort, source text/hash, tree and DAG counts, depths, positional clipping, and available audit status. Bounded-graph numerical outcomes are retained; absent historical per-expression semantic audits are marked explicitly. Scripts, CSV rows, source-only rows, stratified summaries, and input/output digests are supplied under `structural-audit`.

F Experimental result reproducibility

The supplement and trained model states are publicly available at `https://github.com/sahilsinghthefirst/GEML/releases/tag/vericodegen-camera-ready-20261014`. The public release includes all original and new main-study inputs, source pairs, saved prediction scores, results, frozen protocols, analysis code and source snapshots. All 48 current main-study selected/final model states are available: 18 original states in the compact supplement and 30 binding/clipping states in a separate archive. The added model tensors are bit-identical to the archived checkpoints; optimizer and random-number-generator states remain in the author archives. The full historical corpus, historical model weights, and unavailable historical compiled-cost ledger are not supplied. Earlier tiny-control graph payloads and some partial records are omitted, with source pairs and terminal outcomes retained as documented. Lossless containers preserve large ledgers with decoded-byte checksums. The public release README explains how its state extension supplements the older compact ZIP and documents the remaining exclusions.

Use Python 3.12 and the pinned `requirements.txt`; `environment.json` records the original PyTorch 2.11.0/CUDA 12.8/RTX 5090 environment. The compact supplement README gives commands to verify files, recompute metrics and tables from saved scores, run CPU inference from the 18 original states, and retrain. The public release README and `predict_released_states.py` add CPU inference for all 30 binding/clipping states. Their index records the original evaluated checkpoint hashes and training updates. Architecture, optimization, seeds, stopping and model selection are specified in Appendix A. Separate source snapshots preserve the binding and clipping implementations; frozen input and code digests are checked before fresh training.

CPU inference from three representative final states, one per representation over 600 swap pairs, changes no classifications and has maximum logit differences at most 1.05×10^{-5}. The fixed 10^{-6} ranking tie rule is more sensitive: 64/300 historical-EML and 16/300 bounded-EML group rankings change; source rankings do not. Published comparisons use the recorded GPU logits, whose metric replay remains exact. These checks do not establish bitwise hardware independence.

Every added state passed strict loading and a four-pair CPU inference check; the release retains all 30 receipts. This short check verifies loading and input alignment, not full-test replication. The published GPU predictions and their uncertainty calculations remain unchanged.

The numerical ledgers retain individual outcomes, assignments and precisions; the portable re-audit covers every main-study bounded graph at unequal-coordinate witnesses. The structural export supplies the separate 2,700-expression graph ledger and all 250,000 source-only measurement rows, with decoded-byte checksums and reconstruction instructions. Saved-result replay, fresh training and historical reconstruction are distinct checks; none certifies compiler implementations.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction state the three contributions and restrict the learning findings to the tested compilers, populations, and models. Section 7 distinguishes these findings from universal learnability or verification claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 and Appendices B–F discuss grammar restrictions, finite numerical evidence, small training populations, architecture and position effects, post hoc analyses, and unavailable historical artifacts.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 2 sketches the bounded construction; Appendix B states its grammar, interval and exact-arithmetic assumptions, gives the arithmetic identities, and explains induction and sharing. This mathematical argument does not certify the implementation or guarantee floating-point stability.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Appendix F gives the environment, fixed inputs, the 18 included original model mappings, all new frozen recipes and source snapshots, seeds, exact metric and inference commands, expected checks, training specification, and historical structural regeneration route. Appendices A and C specify methods and all seed outcomes. This answer concerns the disclosed reproduction routes, not a claim of fresh historical regeneration or complete historical artifact access; the separate open-access answer records those exclusions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The public release at `https://github.com/sahilsinghthefirst/GEML/releases/tag/vericodegen-camera-ready-20261014` provides code, current data, saved predictions, replay/training commands, and all 48 current main-study model states (18 original and 30 added states). The full historical corpus, historical model weights, and unavailable per-expression compiled-cost ledger remain excluded (Appendix F), so access is incomplete for the full set of reported experiments.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 2 and 5 and Appendix A specify the populations, splits, features, architecture, optimizer, budgets, selection rules, and seeds. Selected and final checkpoints and development-only controls are explicitly distinguished.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Section 5 and Appendix A define sample standard deviations across training seeds and 2,000-draw matched-group bootstrap intervals conditional on those seeds. All individual seeds are retained; the intervals do not claim to capture variation across new training runs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: Appendices A, C and F identify the RTX 5090, run budgets, and active run times, and the supplement records software versions. Complete per-experiment peak memory and project-wide compute, including all historical and preliminary work, are not available in one complete accounting.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

Justification: This synthetic-expression study involves no human participants or personal data; failed experiments and consequential limitations remain disclosed. Section 7 addresses unsafe rewrite acceptance, the LLM-assistance disclosure in the acknowledgments describes the assistance used, and the research data contain no participant records.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 discusses the potential benefit of independent checking and the risk that incorrect equivalence predictions introduce software defects. Practical end-to-end gains and deployment safety are not claimed.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released assets are synthetic symbolic expressions and restricted graph models, not general-purpose generative models or scraped personal data. Section 7 nevertheless requires independent verification before accepting proposed transformations.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: The manuscript credits the original operator and prior learning methods, and the package includes an MIT license for the supplied software. A complete asset-by-asset inventory of third-party licenses and terms is not supplied; therefore we do not claim complete license documentation.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Appendix F and the supplementary README, index, manifests, environment record, and generation configurations document the new code, synthetic inputs, saved models, and replay procedures. Intended scope and excluded artifacts are explicit.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The study uses generated mathematical expressions and computational experiments; it involves no crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human participants, participant data, or participant interventions are involved, so human-subject ethics approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Large language models were used for manuscript refining, figure generation, and as code assistants. Scripts checked the claims against the saved result files and file integrity records used for the final analysis. No LLM is an evaluated model component or source of data or labels, and saved numerical outputs and validation scripts were used to check the reported results. The authors remain responsible for the core manuscript writing, scientific claims, analysis, accuracy, and integrity of the submission.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.